UNIT II

Geometric Display Primitives and Attributes: Geometric display primitives: Points, Lines and Polygons. Point display method – Line drawing: DDA

2D Transformations and Viewing: Transformations-types–matrix representation–Concatenation - Scaling, Rotation, Translation, Shearing, Mirroring. Homogeneous coordinates – Window to view port transformations.

Windowing And Clipping: Point, Lines, Polygons - boundary intersection methods

Geometric Display Primitives

- Graphics programming packages provide functions to describe a scene in terms of these basic geometric structures, referred to as output primitives, and to group sets of output primitives into more complex structures.
- Each output primitives specified with input coordinate data and other information about the way that object is to be displayed. Points and straight line segments are the simplest geometric components of pictures.

Points And Lines

- Point plotting is accomplished by converting a single coordinate position furnished by an application program into appropriate operations for the output device in use.
- With a CRT monitor, for example, the electron beam is turned on to illuminate the screen phosphor at the selected location. How the electron beam is positioned depends on the display technology.
- A random-scan (vector) system stores point-plotting instructions in the display list, and coordinate values in these instructions are converted to deflection voltages that position the electron beam at the screen locations to be plotted during each refresh cycle.
- For a black and white raster system, on the other hand, a point is plotted by setting the bit value corresponding to ***A*** specified screen position within the frame buffer to 1.
- Then, as the electron beam sweeps across each horizontal scan line, it emits a burst of electrons (plots a point) whenever a value of I is encounted in the frame buffer.

- With an RGB system, the frame buffer is loaded with the color codes for the intensities that are to be displayed at the s m n pixel positions.
- Line drawing is accomplished by calculating intermediate positions along the line path between two specified endpoint positions. An output device is then directed to fill in these positions between the endpoints.
- For analog devices, such as a vector pen plotter or a random-scan display, a straight line can be drawn smoothly from one endpoint to the other. Linearly varying horizontal and vertical deflection voltages are generated that are proportional to the required changes in the ***x*** and y directions to produce the smooth line.
- Digital devices display a straight line segment by plotting discrete points between the two endpoints.
- Discrete coordinate positions along the line path are calculated from the equation of the line. For a raster video display, the line color (intensity) is then loaded into the frame buffer at the corresponding pixel coordinates.
- Reading from the frame buffer, the video controller then "plots" the screen pixels. Screen locations are xeferenced with integer values, so plotted positions may only approximate actual Line positions between two specified endpoints.
- The characteristic stair step shape of raster lines is particularly noticeable on systems with low resolution, and we can improve their appearance somewhat by displaying them on high-resolution systems.
- More effective techniques for smoothing raster lines are based on adjusting pixel intensities along the line paths.

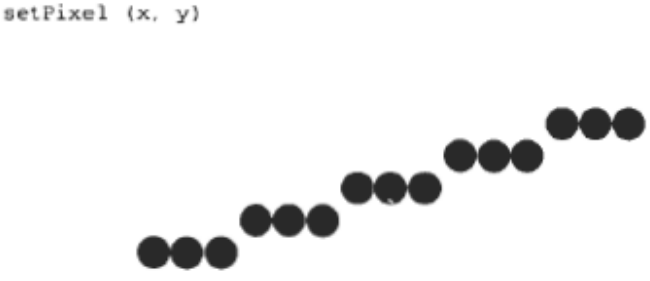


Fig Staircase effect

- Scan lines are numbered consecutively from 0, starting at the bottom of the screen; and pixel columns are numbered from **0,** left to right across each scan line.
- To load a specified color into the frame buffer at a position correspondingto column ***x*** along scan line y, we will assume we have available a low-level procedure of the form.

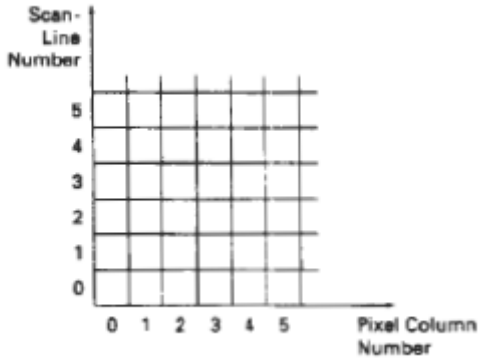


Fig: Pixe1 positions referenced by scanline `number` and column number.

- We sometimes will also want to be able to retrieve the current framebuffer intensity setting for a **specified** location. We accomplish this with the low-level function

`getpixel (x, y)`

Polygons

A **polygon** is a continuous, piecewise linear, closed planar curve.

• A **simple** polygon is non self-intersecting.

• A **regular** polygon is simple, equilateral, and equiangular.

• An **n-gon** is a regular polygon with n sides.

• A polygon is **convex** if, for any two points selected inside the polygon, the line segment between them is completely contained within the polygon.

- A standard output primitive in general graphics packages is a solid-color or patterned polygon area. Other kinds of area primitives are sometimes available, but polygons are easier to process since they have linear boundaries.
- There are two basic approaches to area filling on raster systems. One way to fill an area is to determine the overlap intervals for scan lines that cross the area.

- Another method for area filling is to start from a given interior position and paint outward from this point until we encounter the specified boundary conditions.
- The scan-line approach is typically used in general graphics packages to fill polygons, circles, ellipses, and other simple curves.

Scan-Line Polygon Fill Algorithm

- The scan-line procedure for solid tilling of polygon areas.
- For each scan line crossing a polygon, the area-fill algorithm locates the intersection points of the scan line with the polygon edges.
- These intersection points are then sorted from left to right, and the corresponding frame-buffer positions between each intersection pair are set to the specified fill color.
- In the example of the four pixel intersection positions with the polygon boundaries define two stretches of interior pixels from $x = 10$ to $x = 14$ and from $x = 18$ to $x = 24$. Some scan-line intersections at polygon vertices require special handling.
- A scan line passing through a vertex intersects two edges at that position, adding two points to the list of intersections for the scan line.
- Two scan lines at positions y and y' that intersect edge endpoints. Scan line y intersects five polygon edges. Scan line y', however, intersects an even number of edges although it also passes through a vertex.
- Intersection points along scan line **y'** correctly identify the interior pixel spans. But with scan line y, we need to do some additional processing to determine the correct interior points.

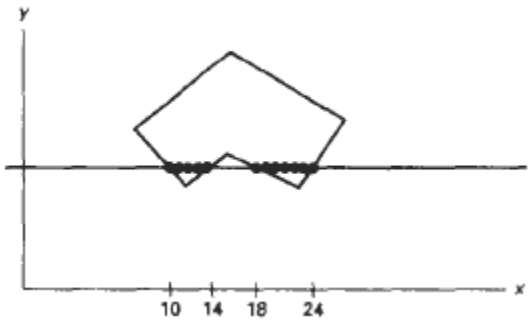
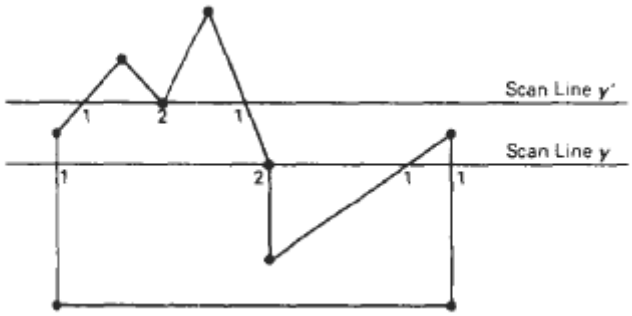


Fig:Interior pixels along a scan line passing through a polygon area

- For scan line y, the two intersecting edges sharing a vertex are on opposite sides of the scan line. But for scan line **y',** the two intersecting edges are both above the scan line. Thus, the vertices that require additional processing are those that have connecting edges on opposite sides of the scan line.
- We can identify these vertices by tracing around the polygon boundary either in clockwise or counterclockwise order and observing the relative changes in vertex y coordinates as we move from one edge to the next.
- If the endpoint y values of two consecutive edges monotonically increase or decrease, we need to count the middle vertex as a single intersection point for any scan line passing through that vertex. Otherwise, the shared vertex represents a local extremum (minimum or maximum) on the **poly**gon boundary, and the two edge intersections with the scan line passing through that vertex can be added to the intersection list.



- Calculations performed in scan-conversion and other graphics algorithmstypically take advantage of various coherence properties of a scene that is to be displayed.

Inside-Outside Tests

- Area-filling algorithms and other graphics processes often need to identify interior regions of objects. So far, we have discussed area filling only in terms of standard polygon shapes. In elementary geometry, a polygon is usually defined as having no self-intersections.
- Examples of standard polygons include triangles, rectangles, octagons, and decagons. The component edges of these objects are joined only at the vertices, and otherwise the edges have no common points in the plane. Identifying the interior regions of standard

polygons is generally a straightforward process. But in most graphics applications, we can specify any sequence for the vertices of a fill area, including sequences that produce intersecting edges.

- Graphics packages normally use either the odd-even rule or the nonzero winding number rule to identify interior regions of an object. We apply the odd-even rule, also called the odd parity rule or the evenodd rule, by conceptually drawing a line from any position P to a distant point outside the coordinate extents of the object and counting the number of edge crossings along the line.
- If the number of polygon edges crossed by this line is odd, then P is an interior point. Otherwise, P is an exterior point. To obtain an accurate edge count, we must be sure that the line path we choose does not intersect any polygon vertices.
- Another method for defining interior regions is the nonzero winding number rule, which.counts the number of times the polygon edges wind around a particular point in the counterclockwise direction. This count is called the winding number.

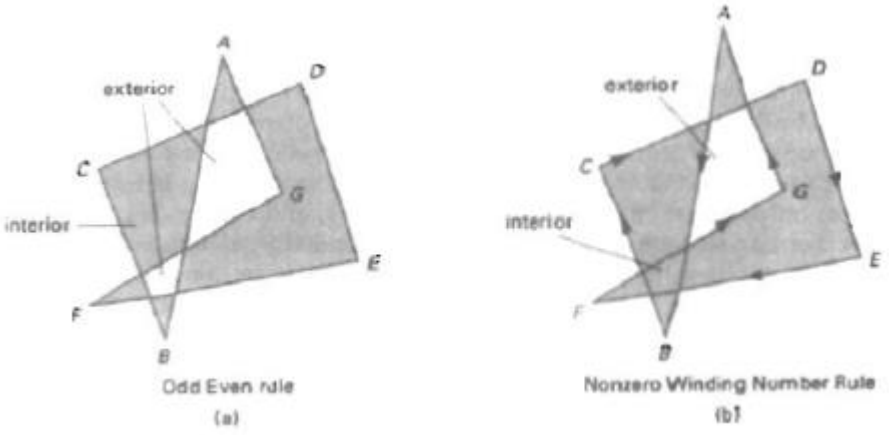


Fig: Identifying interior and exterior regions for a self-intersecting polygon.

Scan-Line Fill of curved boundary areas

- In general, scan-line f i l l of regions with curved boundaries requires more work than polygon filling, side intersection calculation i now involve nonlinear boundaries. For simple curves such as circles or ellipses, performing a scan-line fill is a straightforward process.

Fig:Interior fill of an elliptical arc

- We only need to calculate the two scan-line Intersections on opposite sides of the curve. This is the same as generating pixel positions along the curve boundary, and we can do that with the midpoint method.Then we simply fill in the horizontal pixel spans between the boundary points on Opposite sides of the curve. Symmetries between quadrants (and between octants for circles) are **used** to reduce the boundary calculations.

Line-Drawing Algorithms

The Cartesian slope-intercept equation for a straight line is

$$y = m.x+b \qquad (3.1)$$

With rn representing the slope of the line and b as they intercept. Given that the **two** endpoints of the segment are specified at positions (x_1, y_1) and (x_2, y_2), we can determine values for the slope rn and y intercept b with the following calculations:

$$m = y_2-y_{1/(}x_2-x_{1)} \qquad (3.2)$$

$$b=y_1-m.x_1 \qquad (3.3)$$

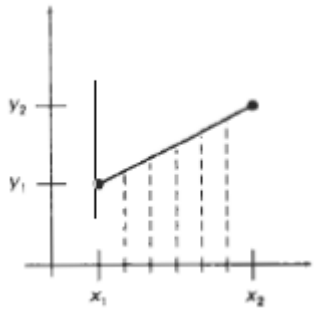
Algorithms for displaying straight lines are based on the line equation 1.1 and the calculations given in Eqn **1**.2 and 1.3. For any given ***x*** interval Δ***x*** along a line, we can compute the corresponding y interval from **Δy** from eqn 1.2 as

$$\Delta y = m \Delta x \qquad (3.4)$$

Eqn(1.4) Similarly, we can obtain the ***x*** interval Δx corresponding to a specified Δy **as**

$$\Delta x = \frac{\Delta y}{m} \qquad (3.5)$$

These equations form the basis for determining deflection voltages in analog devices. For lines with slope magnitudes I ***m*** I < **1, Δx** can be set proportional to a small horizontal deflection voltage and the corresponding vertical deflection is Line-Drawing Algorithms then set proportional to Δy as calculated from Eqn 1.4. For lines whose slopes have magnitudes | ***m*** **|** > **1,** Δy can **be** set proportional to a small vertical deflection voltage with the corresponding horizontal deflection voltage set proportional to **Δx,** calculated from Eq. 1.5. For lines with ***m*** = 1, Δx = Δy and the horizontal and vertical deflections voltages are equal.



In each case, a smooth line with slope m is generated between the specified endpoints.

- On raster systems, lines are plotted with pixels, and step sizes in the horizontal and vertical directions are constrained by pixel separations. That is, we must "sample" a line at discrete positions and determine the nearest pixel to the line at each sampled position. This scan conversion process for straight lines .For a near horizontal line with discrete sample positions along the ***x*** axis.

DDA (The digital differential analyzer)

- The digital differential analyzer **(DDA)** is a scan-conversion line algorithm based on calculating either Δy or Δx ***(eqn 3.4 or 3.5)***
- We sample the line at unit in- Straight line segment with intervals in one coordinate and determine corresponding integer values nearest the five sampling positions along line path for the other coordinate consider first a line with positive slope. If the slope is less than or equal to 1**,** we sample at unit ***x*** intervals ***(***Δx = 1) and compute each successive **y** value as

$$y_{k+1} = y_k + m \qquad (3\text{-}6)$$

- Subscript k takes integer values starting from 1, for the first point, and increases by 1 until the final endpoint is reached. Since m can be any real number between 0 and 1, the calculated y values must be rounded to the n e m t integer. For lines with a positive slope greater than 1, we reverse the roles of ***x*** and y. That is, we sample at unit y intervals (Δy = 1) and calculate each succeeding ***x*** value as

$$x_{k+1} = x_k + \frac{1}{m} \qquad (3\text{-}7)$$

Equations 3.6 and 3.7 are based on the assumption that lines are to be processed from the left endpoint to the right endpoint .If this processing is reversed, so that the starting endpoint is at the right, then either we have Δx = -1 and

$$y_{k+1} = y_k - m \qquad (3\text{-}8)$$

or (when the slope is greater than I) we have Δy = -1 with

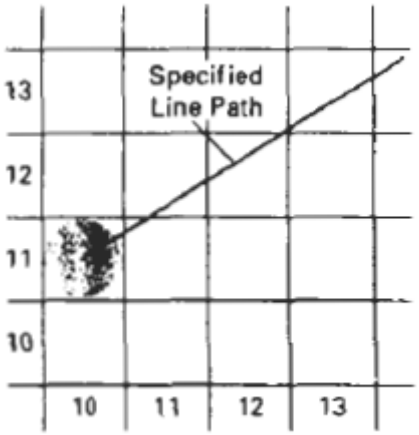
$$x_{k+1} = x_k - \frac{1}{m} \qquad (3.9)$$

Equations 3.6 through 3.9 can also be used to calculate pixel positions along a line with negative slope. If the absolute value of the slope is less than I and the start endpoint is at the left, we set ***Ax*** = 1 and calculate y values with Eqn 3.6. slope, we set Δx = -1 and

Output Primitives obtain y positions from Eq. 3-8. Similarly, when the absolute value of a negative slope is water than 1, we use Δy = -1 and Eq. 3-9 or we use Δy = 1 and Eq. 3-7. This algorithm is summarized in the following procedure, which accepts as input the two end point pixel positions. Horizontal and vertical differences between the endpoint positions *are* assigned to parameters dx and dy. The DDA algorithm is a faster method for calculating pixel positions than the direct use.

BRESENHAM'S LINE ALGORITHM:

An accurate and efficient raster line –generating algorithm, developed by Bresenham. This scan converts lines using only incremental integer calculations that can be adapted to display circles and other curves.



a) Section of a display screen where a straight lins segment is to be plotted, starting from the pixel at column 10 on scan line 11.

From the figure, the vertical axis represents the scan –line positions and the horizontal axis represents the pixel columns.

Consider, endpoint(x, y)

Next position either in (x+1, y) or (x+1, y+1)

TO FIND P_0:

P_0 represents initial pixel position.

At sampling position x_k+1, We label vertical pixel separations from the mathematical line path as d_1 and d_2.

The y coordinate on the mathematical line at pixel column position x_k+1 is calculated as

$$y= m (x_k+1)+ b$$

where,

m is slope

b is constant

TO FIND THE VERTICAL PIXEL SEPARATION:

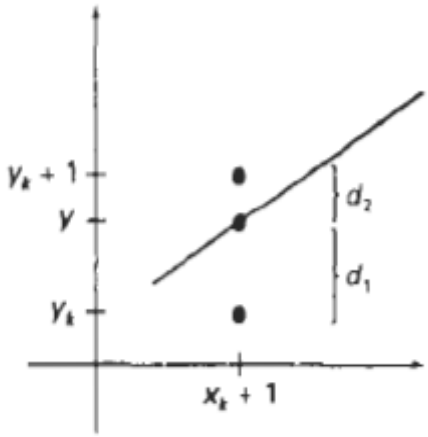


Fig: Distance between the pixel position and the line y co-ordinate at sampling position x_k+1

First pixel separation,

$$d_1 = y - y_k \quad (1)$$

Sub y value in 1

$$d_1 = [m(x_k + 1) + b] - y_k$$

Second pixel separation,

$$d_2 = (y_k + 1) - y \quad (2)$$

Sub y value in 2

$$d_2 = (y_k + 1) - [m(x_k + 1) + b]$$

The difference between these two separations are,

$$d1 - d2 = [[m(x_k + 1) + b] - y_k] - [(y_k + 1) - [m(x_k + 1) + b]] \quad (3)$$

The decision parameter P_k can be calculated by,

$$P_k = \Delta x\ (d1 - d2)$$

$$P_k = \Delta x\,([[m(x_k+1)+b]- y_k]-[(y_k+1)-[m(x_k+1)+b]]) \quad (4)$$

Sub $m=\Delta y/\Delta x$

$$P_k = \Delta x([[\Delta y/\Delta x(x_k+1)+b]- y_k]-[(y_k+1)-[\Delta y/\Delta x(x_k+1)+b]])$$

$$P_k = [[\Delta x.\ \Delta y/\Delta x\ (x_k+1)]+b.\ \Delta x- y_k.\ \Delta x]-[\ \Delta x.(\ y_k+1)-\Delta x.\ \Delta y/\Delta x(x_k+1)-b.\ \Delta x]$$

(Canceling Δx)

$$P_k = \Delta y(x_k+1)+b.\ \Delta x- y_k.\ \Delta x-\Delta x.(y_k+1)+\Delta y\ (x_k+1)+b.\ \Delta x$$

$$P_k = 2\Delta y(x_k+1) - y_k.\ \Delta x- \Delta x.y_k+ 2b.\ \Delta x -\Delta x$$

(Taking Δx as common)

$$P_k = 2\Delta y.(x_k+1)-2\Delta x.y_k+ \Delta x(2b-1)$$

$$P_k = 2\Delta y.\ x_k+2\Delta y -2\Delta x.\ y_k + \Delta x(2b-1) \quad (5)$$

$$P_k = 2\Delta y.\ x_k -2\Delta x.\ y_k +c \quad (6)$$

where $c=2\Delta y+\Delta x(2b-1)$

c is a constant

At step k+1, the decision parameter can be calculated as,

$$P_{k+1} = 2\Delta y.\ x_{k+1} -2\Delta x.\ y_{k+1} +c \quad (7)$$

Subtracting 7 from 6

$$P_{k+1}- P_k = [2\Delta y.\ x_{k+1} -2\Delta x.\ y_{k+1} +c]-[\ 2\Delta y.\ x_k -2\Delta x.\ y_k +c]$$

$$P_{k+1}- P_k = 2\Delta y.\ x_{k+1} -2\Delta x.\ y_{k+1} +c- 2\Delta y.\ x_k +2\Delta x.\ y_k -c$$

Take $x_{k+1} =x_k+1$

$$P_{k+1}- P_k = 2\Delta y.\ (x_k+1) -2\Delta x.\ y_{k+1} - 2\Delta y.\ x_k +2\Delta x.\ y_k$$

$$P_{k+1}- P_k = 2\Delta y.x_k +2\Delta y -2\Delta x.\ y_{k+1} - 2\Delta y.\ x_k +2\Delta x.\ y_k$$

$$P_{k+1} =P_k +2\Delta y -2\Delta x.\ y_{k+1}+2\Delta x.\ y_k$$

$$P_{k+1} =P_k +2\Delta y -2\Delta x(y_{k+1} - y_k) \quad (8)$$

This recursive calculation of decision parameters is performed at each integer x position, starting at the left coordinate endpoint of the line.

The first parameter, P_0 is evaluated from equation (5)

$$(5) \Rightarrow P_k = 2\Delta y.\, x_k + 2\Delta y - 2\Delta x.\, y_k + \Delta x(2b-1)$$

Sub k=0

$$P_0 = 2\Delta y.\, x_0 + 2\Delta y - 2\Delta x.\, y_0 + \Delta x(2b-1)$$

Put $x_0 = 0$, $y_0 = 0$

$$P_0 = 2\Delta y(0) + 2\Delta y - 2\Delta x(0) + \Delta x(2b-1) \qquad (9)$$

Here b is constant, So neglect $2b\Delta x$ term from (9)

$$P_0 = 0 + 2\Delta y - 0 - \Delta x$$

$$P_0 = 2\Delta y - \Delta x \text{ [Initial pixel position]}$$

Algorithm :

1. Input the two line endpoints and store the left endpoint in (x_0, y_0).
2. Load (x_0, y_0) into the frame buffer; that is, plot the first point.
3. Calculate constants Δx, Δy, $2\Delta y$, and $2\Delta y - 2\Delta x$, and obtain the starting value for the decision parameter as

$$p_0 = 2\Delta y - \Delta x$$

4. At each x_k along the line, starting at $k = 0$, perform the following test: If $p_k < 0$, the next point to plot is $(x_k + 1, y_k)$ and

$$p_{k+1} = p_k + 2\Delta y$$

   Otherwise, the next point to plot is $(x_k + 1, y_k + 1)$ and

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

5. Repeat step 4 Δx times.

2D TRANSFORMATIONS AND VIEWING

- Animations are produced by moving the "camera" or the objects in a scene along animation paths.
- Changes in orientation, size, and shape are accomplished with geometric transformations that alter the coordinate descriptions of objects.
- The basic geometric transformations are translation, rotation, and scaling. Other transformations that are often applied to objects include reflection and shear.

Translation

A translation is applied to an object by repositioning it along a straight-line path from one coordinate location to another. We translate a two-dimensional point by adding translation distances, ***f,*** and t,, to the original coordinate position **(x,** y) to move the point to a new position

$$(x', y') \quad x' = x + t_x, \quad y' = y + t_y \qquad (5.1)$$

The translation distance pair (t_x, t_y) is called a translation vector or shift vector. We can express the translation equations 5-1 as a single matrix equation by using column vectors to represent coordinate positions and the translation vector

$$\mathbf{P} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \mathbf{P}' = \begin{bmatrix} x_1' \\ x_2' \end{bmatrix}, \quad \mathbf{T} = \begin{bmatrix} t_x \\ t_y \end{bmatrix} \qquad (5\text{-}2)$$

$$\mathbf{P}' = \mathbf{P} + \mathbf{T} \qquad (5\text{-}3)$$

This allows us to write the two-dimensional translation equations in the matrix form:

- Sometimes matrix-transformation equations are expressed in terms of coordinate row vectors instead of column vectors.
- In this case, we would write the matrix representations as **P** = ***[x y]*** and T =(t_x,t_y)**.** Since the column-vector representation for a point is standard mathematical notation, and since many graphics packages, for example, **GKS** and PHIGS, also use the column-vector representation, we will follow this convention.

- Translation is a **rigid-body transformation** that moves objects without deformation. That is, every point on the object is translated by the same amount. **A** straight Line segment is translated by applying the transformation equation

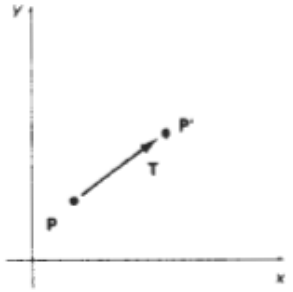
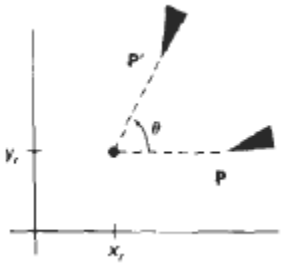


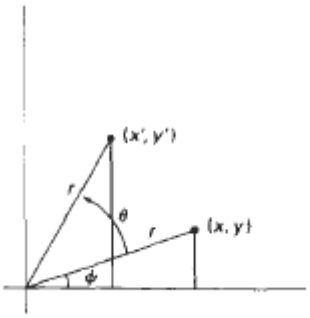
Fig Translating a point from position P to position P' with translation vector T

- Polygons are translated by adding the translation vector to the coordinate position of each vertex and regenerating the polygon using the new set of vertex coordinates and the current attribute settings.

Rotation

A two-dimensional rotation is applied to an object by repositioning it along a circular path in the xy plane. To generate a rotation, we specify a rotation angle 0 and the position (x_1, y_1) of the rotation point (or pivot point) about which the object is to be rotated Positive values for the rotation angle define counterclockwise rotations about the pivot point, and negative values rotate objects in the clockwise direction.





- This transformation can also be described as a rotation about a rotation axis that is perpendicular to the xy plane and passes through the pivot point.
- We first determine the transformation equations for rotation of a point position P when the pivot point is at the coordinate origin.
- The angular and coordinate relationships of the original and transformed point positions , **r** is the constant distance of the point from the origin, angle Φ is the original angular position of the point from the horizontal, and t3 is the rotation angle. θUsing standard trigonometric identities, we can express the transformed coordinates in terms of angles θ and Φ as

$$x' = r\cos(\phi + \theta) = r\cos\phi\cos\theta - r\sin\phi\sin\theta$$
$$y' = r\sin(\phi + \theta) = r\cos\phi\sin\theta + r\sin\phi\cos\theta \qquad (5\text{-}4)$$

The original coordinates of the point in polar coordinates are

$$x = r\cos\phi, \qquad y = r\sin\phi \qquad (5\text{-}5)$$

Substituting expressions 5-5 into 5-4, we obtain the transformation equations for rotating a point at position ***(x, y)*** through an angle Φ about the origin:

$$x' = x\cos\Phi - y\sin\Phi$$
$$\mathbf{y'} = x\sin\Phi + y\cos\Phi \qquad (5.6)$$

With the column-vector representations 5-2 for coordinate positions, we can write the rotation equations in the matrix form:

$$P' = R \cdot P \qquad (5.7)$$

where the rotation matrix is

$$\mathbf{R} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \qquad (5\text{-}8)$$

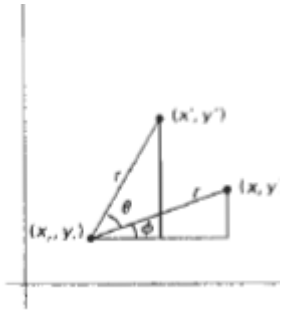
When coordinate positions are represented as row vectors instead of column vectors, the matrix product in rotation equation 5-7 is transposed so that the transformed row coordinate vector Ix' y'l iscalculated as

$$\begin{aligned} \mathbf{P}'^{T} &= (\mathbf{R} \cdot \mathbf{P})^{T} \\ &= \mathbf{P}^{T} \cdot \mathbf{R}^{T} \end{aligned}$$

where **PT = (x y],** and the transpose RT of matrix R is obtained by interchanging rows and columns. For a rotation matrix, the transpose **is** obtained by simply changing the sign of the sine terms.

- Rotation of a point about an arbitrary pivot position I using the trigonometric relationships in this figure, we can generalize Eqn. 5-6 to obtain the transformation equations for rotation of a point about any specified rotation position **(x_1,y_1)**

$$\begin{aligned} x' &= x_r + (x - x_r)\cos\theta - (y - y_r)\sin\theta \\ y' &= y_r + (x - x_r)\sin\theta + (y - y_r)\cos\theta \end{aligned} \qquad (5\text{-}9)$$



As with translations, rotations are rigid-body transformations that move objects without deformation. Every point on an object is rotated through the same angle.

Scaling

A scaling transformation alters the size of an object. This operation can be carried out for polygons by multiplying the coordinate values ***(x,*** y) of each vertex by scaling factors s, and s, to produce the transformed coordinates **(x',** y'):

$$x' = x \cdot s_x, \qquad y' = y \cdot s_y \tag{5-10}$$

Scaling factors, scales objects in the **x** direction, while s_y scales in they direction.

The transformation equations 5-10 can also be written in the matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \tag{5-11}$$

or

$$P' = S \cdot P \tag{5-12}$$

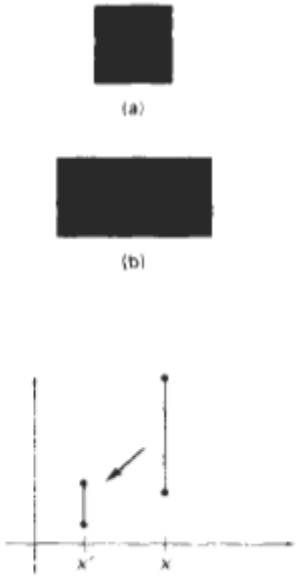


Fig: A line scaled using sx-sy = 0.5 is reduced in size and moved closer *to* the co-ordinate origin.

- We can control the location of a scaled object by choosing a position, called the fixed point, that is to remain unchanged after the scaling transformation. Coordinates for the fixed point (x1, y,) can be chosen as one of the vertices, the object centroid, or any other position.

- **A** polygon is then scaled relative to the fixed point by scaling the distance from each vertex to the fixed point. For a vertexwith coordinates **(x,y)** the scaled co-ordinates***(x ' ,*** y ') are calculated as

$$x' = x_f + (x - x_f)s_x, \qquad y' = y_f + (y - y_f)s_y$$

- We can rewrite these scaling transformations to separate. the mdtiplicative and additive terms:

$$x' = x \cdot s_x + x_f(1 - s_x)$$
$$y' = y \cdot s_y + y_f(1 - s_y)$$

- where the additive terms $x_f(1 - s_x)$ and $y_f(1 - \mathbf{s_y})$ are constant for all points in the object.

Shearing

- **A** transformation that distorts the shape of an object such that the transformed shape appears as if the object were composed of internal layers that had been to slide over each other is called a shear.
- Two common shearing transformations are those that shift coordinate **w** values and those that shift y values.
- An x-direction shear relative to the x axis is produced with the transformation Matrix

$$\begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad (5\text{-}53)$$

which transforms coordinate positions as

$$x' = x + sh_x \cdot y, \qquad y' = y \qquad (5\text{-}54)$$

Any real number can be assigned to the shear parameter s_h,. A coordinate position ***(.u,*** y) is then shifted horizontally by an amount proportional to its distance (y value) from the x axis (y = **0).** Setting s_h, to 2, for example, changes the square in Fig. 5-23 into a parallelogram. Negative values for s_h, shift coordinate positions to the left.

We can generate x-direction shears relative to other reference lines with

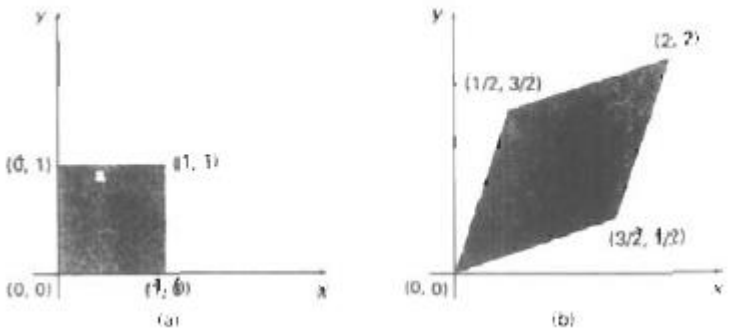
$$\begin{bmatrix} 1 & sh_x & -sh_x \cdot y_{ref} \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Concatenation

Matrix multiplication is associative. For any three matrices, A, B, and C, the matrix product A.B. C can be performed by first multiplying A and B or by first multiplying B and C:

$$A.B.C= (A.B).C=A.(B.C)$$

Therefore, we can evaluate matrix products using either a left-to-right or a right to- left associative grouping. On the other hand, transformation products may not be commutative: The matrix product A. B is not equal to B - A, in general.



- This means that if we want to translate and rotate an object, the order in which the composite matrix is evaluated.
- For some special cases, such as a sequence of transformations all of the same kind, the multiplication of transformation matrices is commutative. As an example, two successive rotations could be performed in either order and the final position would be the same. This commutative property holds also for two successive translations or two successive scalings.
- Another commutative pair of operations is rotation and uniform scaling (s_x=s_y)

Matrix Representations and Homogeneous Co-ordinates

- Many graphics applications involve sequences of geometric transformations. An animation, for example, might require an object to be translated and rotated at each increment of the motion.
- In design and picture construction applications, we perform translations, rotations, and scaling to tit the picture components into their proper positions.
- Here we consider how the matrix representations dis- Matrix Representations can be reformulated so that such transformation sequences can be efficiently processed.

$$\mathbf{P}' = \mathbf{M}_1 \cdot \mathbf{P} + \mathbf{M}_2$$

- With coordinate positions P and P' represented as column vectors. Matrix M_1 is a 2 by 2 array containing multiplicative factors, and **M,** is a two-element column matrix containing translational terms. For translation, M_1 is the identity matrix.
- For rotation or scaling, **M2** contains the translational terms associated with the pivot point or scaling fixed point.

- To produce a sequence of transformations with these equations, such as scaling followed by rotation then translation, we must calculate the transformed coordinate's one step at i_1 time. First, coordinate positions are scaled, then these scaled coordinates are rotated, and finally the rotated coordinates are translated.
- ***A*** more efficient approx would be to combine the transformations so that the final coordinate positions are obtained directly from the initial coordinates, thereby eliminating the calculation of intermediate coordinate values.
- We can combine the multiplicative and translational terms for two-dimensional geometric transformations into a single nratrix representation by expanding the 2 by 2 matrix representations to 3 by 3 matrices. This allows us to express all transformation equations as matrix multiplications, providing that we also expand the matrix representations for coordinate positions.
- To express any two-dimensional transformation as a matrix multiplication, we represent each Cartesian coordinate position ***(x,y)*** with the homogeneous coordinate triple ***(x_h,*** y_h, *h),*where

$x=x_h/h$,$y=y_h/h$

- A general homogeneous coordinate representation can also be written as *(hx.h.y.h).* For two-dimensional geometric transformations, we can choose the homogenous parameter h to be any nonzero value.
- Thus, there is an infinite number of equivalent homogeneous representations for ea ch coordinate point ***(x,*** y). **A** convenient choice is simply to set h = 1. Each two dimensional position is then represented with homogeneous coordinates ***(x,*** y, 1).
- Other values for parameter h are needed, for example, in matrix formulations of three dimensional viewing Transformations
- The term homogeneous coordinates used in mathematics to refer to the effect of this representation on Cartesian equations.
- Then a Cartesian point ***(x,* y)** is converted to a homogeneous representation ***(x$_h$,*** y_h, h), equations containing ***x*** and y**,** such as *f(x,* y) = 0, become homogeneous equations in the three parameters **x_h, y_h,** and **h.**'This just means that if each of the three parameters is replaced by any value ***n*** times that parameter, the value can he factored out of the equations.
- Expressing positions in homogeneous coordinates allows us to represent all geometric transformation equations as matrix multiplications. Coordinates are represented with three-element column vectors, and transformation operations

Two-Dimensional **Geometric** are written as 3 bv 3 matrices. For translation. we have

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

which we can write in the abbreviated form

$$\mathbf{P}' = \mathbf{T}(t_x, t_y) \cdot \mathbf{P}$$

with $T(t_x, t_y)$as the 3 by 3 translation matrix .

- The inverse of the translation matrix is obtained by replacing the translation parameters 1, and 1**,** with their negatives: - ***tx,*** and ***-ty,.***

Similarly, rotation transformation equations about the coordinate origin are now written as

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

or as

$$\mathbf{P}' = \mathbf{R}(\theta) \cdot \mathbf{P}$$

- The rotation transformation operator $\mathbf{R}(\theta)$1s the 3 by 3 matrix rotation parameter θ. We get the inverse rotation matrix when θ is replaced with - θ.

- Finally, a scaling transformation relative to the coordinate origins now expressed as the matrix multiplication

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

or $\mathbf{P}' = \mathbf{S}(s_x, s_y) \cdot \mathbf{P}$

Window to view port transformations.

- Once object descriptions have been transferred to the viewing reference frame, we choose the window extents in viewing coordinates and select the viewport limits in normalized coordinates Object descriptions are then transferred to normalized device coordinates.
- using a transformation that maintains the same relative placement of objects in normalized space as they had in viewing coordinates.

- If a coordinate position is at the center of the viewing window, for instance, it will be displayed at the center of the viewport.

- The window-to-viewport mapping. **A** point at position ***(xw,yw)*** in the window 1s mapped into position ***(xv,*** yv) in the associated viewport.

To maintain the same relative placement in the viewport as in the window,

$$\frac{xv - xv_{min}}{xv_{max} - xv_{min}} = \frac{xw - xw_{min}}{xw_{max} - xw_{min}}$$

$$\frac{yv - yv_{min}}{yv_{max} - yv_{min}} = \frac{yw - yw_{min}}{yw_{max} - yw_{min}}$$

Solving these expressions for the viewport position (xv, yv), we have

$$xv = xv_{min} + (xw - xw_{min})sx$$

$$yv = yv_{min} + (yw - yw_{min})sy$$

where the scaling factors are

$$sx = \frac{xv_{max} - xv_{min}}{xw_{max} - xw_{min}}$$

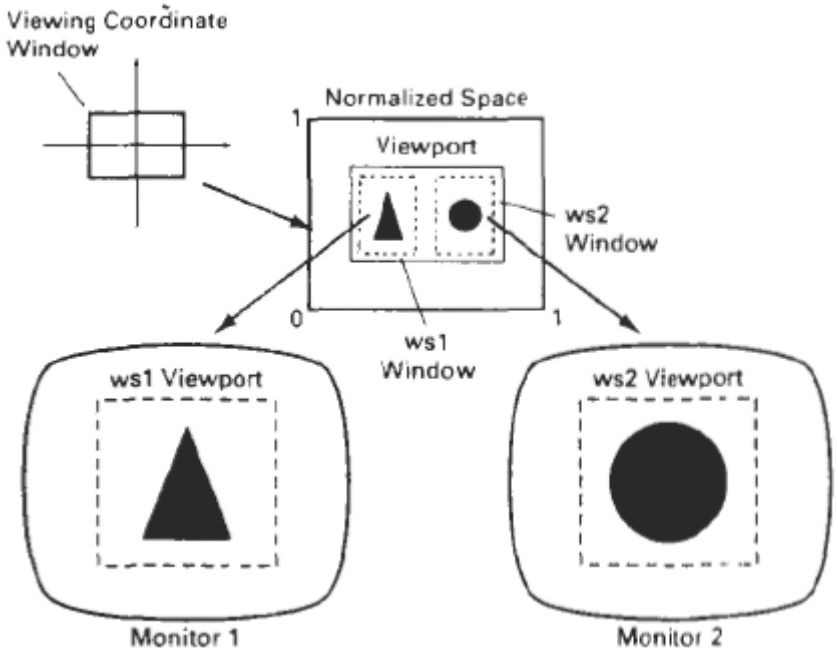
$$sy = \frac{yv_{max} - yv_{min}}{yw_{max} - yw_{min}}$$

A set of transformations that converts the window area into the viewport area. This conversion is performed with the following sequence of transformations:

1. Perform a scaling transformation using a fixed-point position of ***(xw, yw,,,)*** that scales the window area to the size of the **viewpoint.**

2. Translate the scaled window area to the position of the viewport. Relative proportions of objects are maintained if the scaling factors are the same ***(sx = sy).***

- Otherwise, world objects will be stretched or contracted in either the ***x*** or ***y*** direction when displayed on the output device.
- Character strings can be handled in two ways when they are mapped to a viewport. The simplest mapping maintains a constant character size, even though the viewport area may be enlarged or reduced relative to the window.
- It is method would be employed when text is formed with standard character fonts that cannot be changed. In systems that allow for changes in character size, string definitions can be windowed the same as other primitives.
- For characters formed with line segments, the mapping to the viewport can be carried out as a sequence of line transformations. From normalized coordinates, **object** descriptions are mapped to the various display devices.
- Any number of output devices can be open in a particular application, and another window-to-viewport transformation can be performed for each open output device. This mapping, called the workstation **transformation**



Accomplished by selecting a window area in normalized space and a viewport area in the coordinates of the display device. With the workstation transformation, we gain some additional control over the positioning of parts of a scene on individual output devices.

Windowing And Clipping:

Point Clipping

Assuming that the clip window is a rectangle in standard position, we save a point P = ***(x, y)*** for display if the following inequalities are satisfied:

$$xw_{min} \le x \le xw_{max}$$

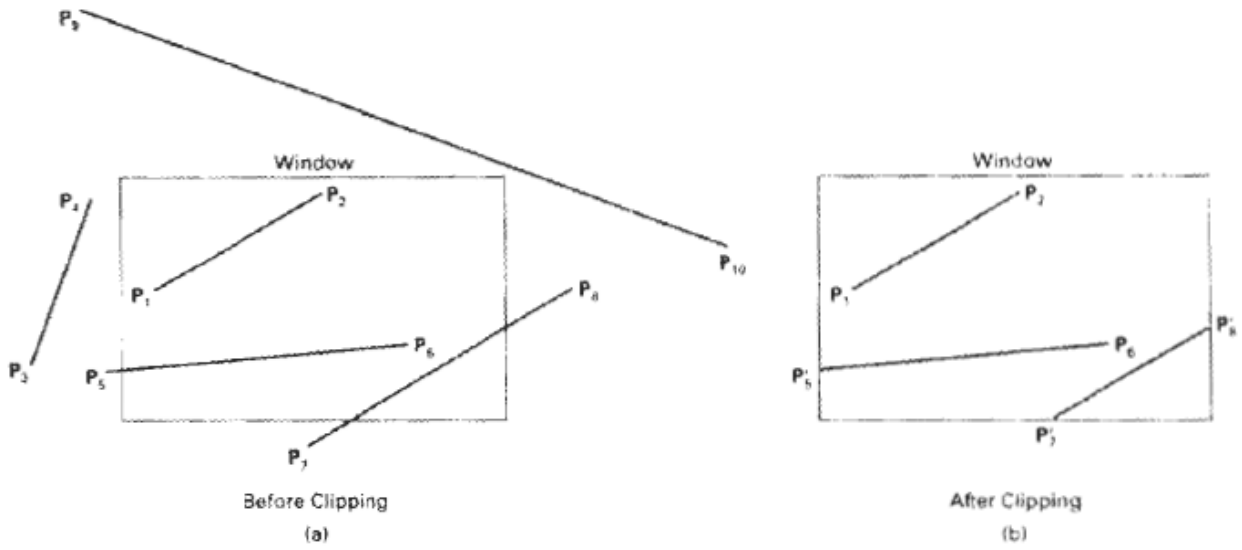
$$yw_{min} \le y \le yw_{max}$$

where the edges of the clip window $(xw_{min}, xw_{max}, yw_{min}, yw_{max})$ can be either the world-coordinate window boundaries or viewport boundaries.

- If any one of these four inequalities is not satisfied, the point is clipped (not saved for display). Although point clipping is applied less often than line or polygon clipping, some .applications may require a point clipping procedure.
- For example, point clipping can be applied to scenes involving explosions or sea foam that are modeled with particles (points) distributed in some region of the scene.

Line Clipping

- A line clipping procedure involves several parts. First, we can test a given line segment to determine whether it lies completely inside the clipping window. If it does not, we try to determine whether it lies **com**pletely outside the window.
- Finally, if we cannot identify a line as completely inside or completely outside, we must perform intersection calculations with one or more clipping boundaries. We process lines through the "inside-outside" tests by checking the line endpoints.
- A line with both endpoints inside all clipping boundaries, such as the line from P1, to **P2** saved



- All other lines cross one or more clipping boundaries, and may require calculation of multiple intmstution points. To minimize calculations, we try to devise clipping algorithms that can efficiently identify outside lines and reduce intersection calculations.For a line segment with endpoints (x_1, y_l) and (x_2, Y_2) and one or both endpoints outside the clipping rectangle, the parametric reprcsentation.

$$x=x_1+u(x_2-x_1)$$

$$y=y_1+u(y_2-y_1)$$

$$0\leq u\leq 1$$

u-> clipping boundary co-ordinates

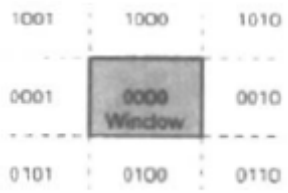
- Clipping line segments with these parametric **tests** requires a good deal of computation, and faster approaches to clipping are possible.

Cohen-Sutherland Line clipping

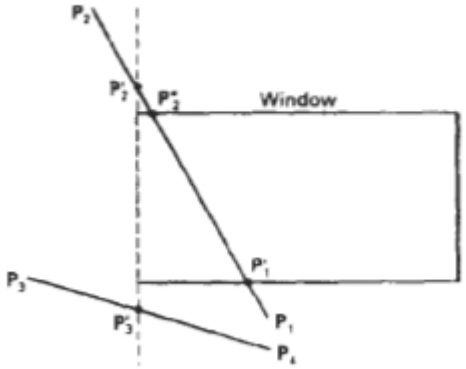
- This is one of the oldest and most popular line-clipping procedures. Generally, the method speeds up the processing of line segments by performing initial tests that reduce the number of intersections that must he calculated. Every line endpoint in a picture is assigned a four-digit binary code, called a region code, that identifies the location of the point relative to the boundaries of the clipping rectangle.

- Each bit position in the region code is used to indicate one of the four relative coordinate positions of the point with respect to the clip window: to the left, right, top, or bottom. By numbering the bit positions in the region code as 1 through 4 from right to left, the coordinate regions can be correlated with the bit positions as

bit 1**:** left
bit 2: right
bit 3: below
bit 4: above



- A value of 1 in any bit position indicates that the point is in that relative position; otherwise, the bit position is set to 0. If a point is within the clipping rectangle, the region code is 0000. A point that is below and to the left of the rectangle has a region code of 0101.
- Starting with the bottom endpoint of the line from **P_1,** to **P_2,**



- we check P, against the left, right, and bottom boundaries in turn and find that this point is below the clipping rectangle. We then find the intersection point P_1 with the bottom boundary and discard the line section from P_1 to P_1'**.** The line now has been reduced to the section from **P_1'** to **P, .**

- Since **P_2,** is outside the clip window, we check this endpoint against the boundaries and find that it is to the left of the window. Intersection point **P_2'** is calculated, but this point is above the window. So the final intersection calculation yields **P_2",** and the line from P_1' to **P_1'';** is saved.
- This completes processing for this line, so we save this part and go on to the next line. Point **P3** in the next line is to the left of the clipping rectangle, so we determine the intersection **P,** and eliminate the line section from **P_3** to **P_3'** By checking region codes for the line section from **P_3** 'to **P_4,** we find that the remainder of the line is below the clip window and can be discarded also.

Liang-Barsky Line Clipping

- Faster line clippers have been developed that are based on analysis of the parametric equation of a line segment, which we can write in the form

$$x = x_1 + u\Delta x$$
$$y = y_1 + u\Delta y, \qquad 0 \le u \le 1$$

where **$\triangle x$** .= **x_2** – **x_1** and **$\triangle y$** = **y_2** – **y_1.** Using these parametric equations, Cyrus and Beck developed an algorithm that is generally more efficient than the Cohen-Sutherland algorithm. Later, Liang and Barsky independently devised an even faster parametric line-clipping algorithm.

$$xw_{min} \le x_1 + u\Delta x \le xw_{max}$$
$$yw_{min} \le y_1 + u\Delta y \le yw_{max}$$

Each of these four inequalities can be expressed as

where parameters p and q are defined as

$$p_1 = -\Delta x, \qquad q_1 = x_1 - xw_{min}$$
$$p_2 = \Delta x, \qquad q_2 = xw_{max} \quad x_1$$
$$p_3 = -\Delta y, \qquad q_3 = y_1 - yw_{min}$$
$$p_4 = \Delta y, \qquad q_4 = yw_{max} \quad y_1$$
$$up_k \le q_k, \qquad k = 1, 2, 3, 4$$

- In general, the Liang-Barsky algorithm is more efficient than the Cohen-Sutherland algorithm, since intersection calculations are reduced.
- Both the Cohen-Sutherland and the Liang-Barsky algorithms can be extended to three-dimensional clipping.

Nicholl-Lee-Nic-holl Line Clipping

- By creating more regions around the clip window, the Nicholl-Lee-Nicholl (or NLN) algorithm avoids multiple clipping of an individual line segment. In the Cohen-Sutherland method, for example, multiple intersections may be calculated along the path of a single line before an intersection on the clipping rectangle is located or the line is completely repcted.
- These extra intersection calculations are eliminated in the NLN algorithm by carrying out more region testing before intersection positions are calculated. Compared to both the Cohen-Sutherland and the Liang-Barsky algorithms, the Nicholl-Lee-Nicholl algorithm performs fewer comparisons and divisions. The trade-off is that the NLN algorithm can only be applied to two-dimensional dipping, whereas both the Liang-Barsky and the Cohen-Sutherland methods are easily extended to three-dimensional scenes.

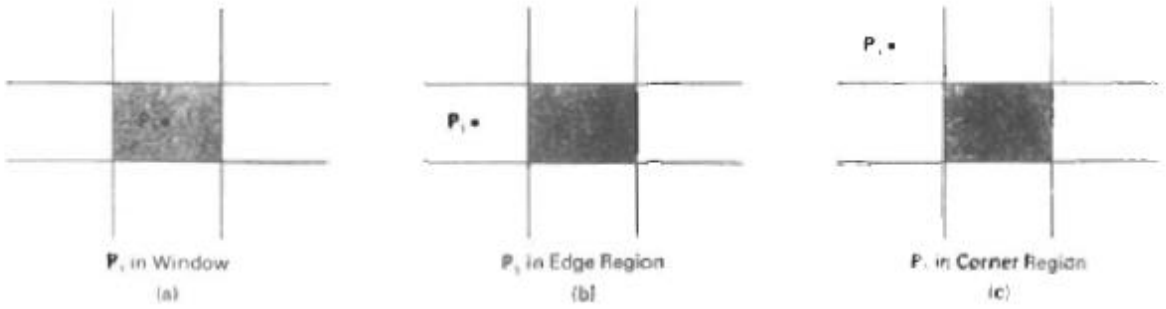
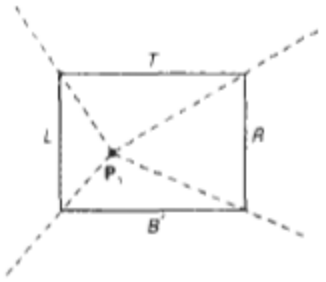
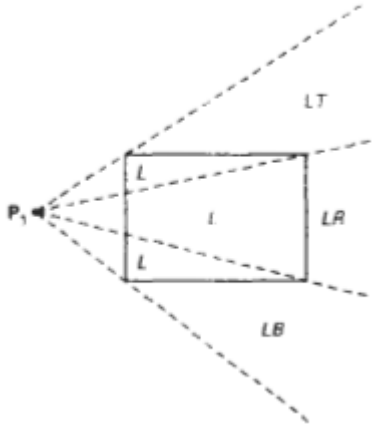


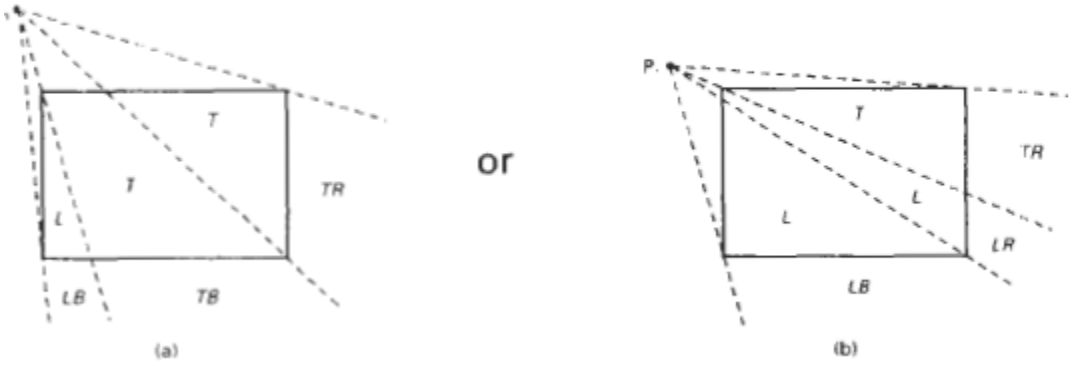
Fig:Three possible **positions** for a line endpoint **P,** in the NLN line-clipping algorithm.

For example, the region directly above the clip window can be transformed to the region left of the clip window using a reflection





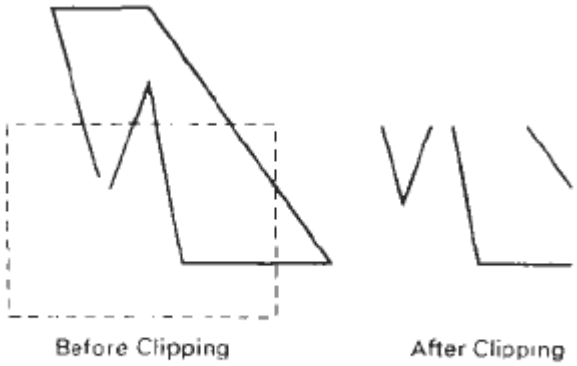
In this case, we have the two possibilites shown, depending on the position of **P,** relative to the top left corner of the window. If **P,** is in one of the regions T, L, TR, ***T8,*** LR, or LB, this determines a unique clipwindow edge for the intersection calculations. Otherwise, the entire line is rejected.

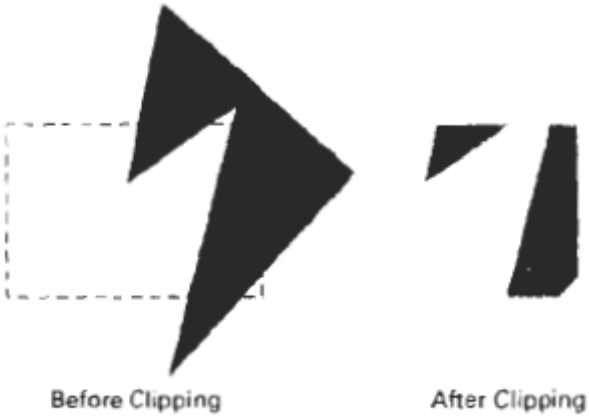


Polygon Clipping

- **A** polygon boundary processed with a line clipper may be displayed as a series of unconnected line segments depending on the orientation of the polygon to the clipping window.

- What we really want to display is a bounded area after clipping For polygon clipping, we require an algorithm that will generate one or more closed areas that are then scan converted for the appropriate area fill. The output of a polygon clipper should be a sequence of vertices that defines the clipped polygon boundaries

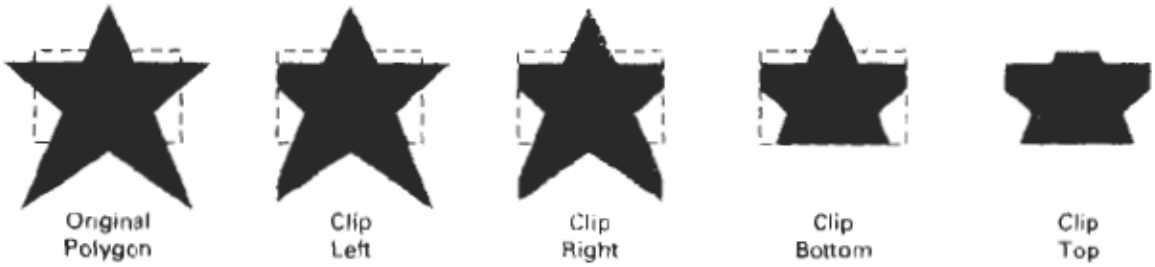




Sutherland-Hodgeman polygon Clipping

- We can correctly clip a polygon by processing the polygon boundry as a whole against each window edge. This could be accomplished by processing all polygon vertices against each clip rectangle boundary in turn.

- Beginning with the initial set of polygon vertices, we could first clip the polygon against the left rectangle boundary to produce a new sequence of vertices. The new set of vertices could then k successively passed to a right boundary clipper, a bottom boundary clipper, and a top boundary clipper

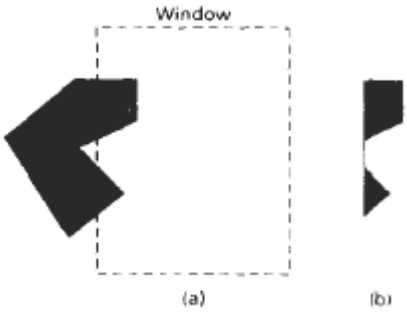


- There are four possible cases when processing vertices in sequence around the perimeter of a polygon. As each pair of adjacent polygon vertices is passed to a window boundary clipper, we make the following tests:
- **(1)** If the first vertex is outside the window boundary and the second vertex is inside, both the intersection point of the polygon edge with the window boundary and the second vertex are added to the output vertex list.
- **(2)** If both input vertices are inside the window boundary, only the second vertex is added to the output vertex list.
- **(3)** The first vertex is inside the window boundary and the second vertex is outside, only the edge intersection with the window boundary is added to the output vertex list.
- **(4)** If both input vertices are outside the window boundary, nothing is added to the output list.

Wciler-Atherton Polygc:n Clipping

- Here, the vertex-processing procedures for window boundaries are modified so that concave polygons are displayed correctly. This clipping procedure was developed as a method for identifying visible surfaces, and so it can be applied with arbitrary polygon-clipping regions.
- The basic idea in this algorithm is that instead of always proceeding around the polygon edges as vertices are processed, we sometimes want to follow the window boundaries.

Which path we follow depends on the polygon-processing direction (clockwise or counterclockwise) and whether tile pair of polygon vertices currently being processed represents an outside-to-inside pair or an inside-



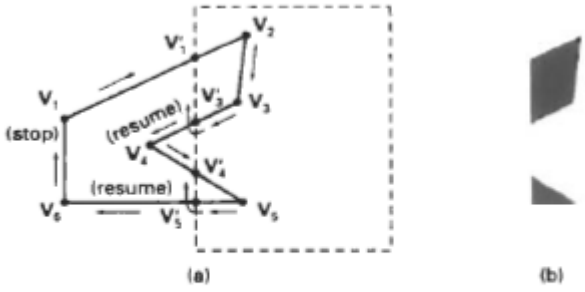


Figure 6-25
Clipping a concave polygon (a) with the Weiler–Atherton algorithm generates the two separate polygon areas in (b).

to-outside pair. For clockwise processing of polygon vertices, we use the following rules:

- For an outside-to-inside pair of vertices, follow the polygon boundary.
- For an inside-to-outside pair of vertices,. follow the window boundary in a clockwise direction.

Other Polygon-Clipping Algorithms

- Various parametric line-clipping methods have also been adapted to polygonclipping. And they are particularly well suited for clipping against convex polygon-clipping windows.

Clipping a polygon by determining the intersection of two polygon areas

- The Liang-Barsky Line Clipper, for example, can be extended to polygon clipping with a general approach similar to that of the Sutherland-Hodgeman method. Parametric line representations are used to processpolygon edges in order around the polygon perimeter using region-testing procedures similar to those used in line clipping.